

Helix OTA — Stack Research: Mender

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Evidence-based research note on Mender (mendersoftware) as a candidate wrapped OTA engine for Helix. Covers client+server architecture, A/B + delta, phased deployments, the reality of Android support, client language, server/client coupling, and the open-source vs commercial licensing split. Sourced from Mender official docs, mender.io marketing/pricing pages, and the GitHub client repo, all consulted 2026-06-07.
Issues	Mender has no Android support (official docs list Debian-family Linux, Yocto, Buildroot/OpenWRT, and Zephyr/MCU only — Android is not mentioned). Most fleet-orchestration features Helix needs (phased rollouts, dynamic grouping, RBAC, delta generation) are paid/Enterprise-only, not in the open-source server. This makes Mender a poor fit for Helix's Android-15-first + native-A/B + custom-Go-control-plane mandate.
Issues summary	Android-15 fit is effectively zero; differentiating control-plane features are behind the commercial license.
Fixed	initial research
Fixed summary	All load-bearing claims verified against primary sources or explicitly flagged UNVERIFIED.
Continuation	Feed into research/ota_landscape_report.md and ADR-0001 (engine selection). Compare head-to-head with the hawkBit note. If Helix keeps the

Field	Value
	native-A/B + custom-Go control plane, Mender is a reference/anti-pattern source, not a wrap target.

Table of contents

- [§1. Summary & verdict](#)
- [§2. What Mender is](#)
- [§3. Client architecture](#)
- [§4. Server architecture](#)
- [§5. A/B updates & delta updates](#)
- [§6. Phased deployments & fleet orchestration](#)
- [§7. Android support — the reality](#)
- [§8. Client language & coupling](#)
- [§9. Licensing: open source vs commercial](#)
- [§10. Fit for Helix](#)
- [§11. Confidence & open questions](#)
- [§12. Sources consulted](#)

§1. Summary & verdict

Mender is a mature, production-grade OTA solution for **embedded Linux** (and, more recently, Zephyr/MCU). Its client-server design, dual-A/B-rootfs robustness model, Update Modules framework, and delta/phased-rollout capabilities are exactly the feature set Helix cares about — **but for the wrong platform**. Mender has no Android support, and the orchestration features that overlap most with Helix’s control plane (phased rollouts, dynamic grouping, RBAC, server-side delta generation) are gated behind paid Professional/Enterprise tiers, not the open-source server.

For Helix (Android 15 first, native A/B via AOSP `update_engine`, custom Go control plane) Mender is **not a viable wrap target**. It is most useful as a **design reference** for the control-plane API shape, the Update Modules abstraction, and the robustness/rollback model. Overall confidence: **high** on the disqualifying facts (Android, licensing split); **medium** on exact feature-to-tier mapping (marketing pages shift).

§2. What Mender is

Mender (by Northern.tech / mendersoftware) is an open-source, end-to-end OTA update manager for IoT and embedded Linux devices. It comprises two halves:

- **Mender Server** — central management of deployments, device inventory, and artifacts, exposed via web UI + REST APIs.
- **Mender Client** — a device-side daemon (or standalone CLI) that polls the server, downloads artifacts, and installs them robustly.

It is positioned for embedded Linux fleets (industrial IoT, gateways, medical/automotive embedded). Source: [Introduction](#) | [Mender docs](#), [How Mender works](#).

§3. Client architecture

- The client runs in **managed mode** (a daemon that continuously polls the server for updates) or **standalone mode** (manually triggered via CLI; supports fully offline updates from a local web server or USB).
- The modern client is split into core subcomponents **mender-auth** / **mender-authd** (authentication) and **mender-update** / **mender-updated** (update execution). Earlier versions shipped a single `mender-client` binary/service. Source: [Client installation overview](#), [Standalone deployment](#).
- **Update Modules framework**: rather than hardcoding install methods, the client invokes **independent executables** in `/usr/share/mender/modules/v3` with a defined parameter/state interface. This lets Mender support arbitrary packaging formats — full rootfs images, application/package updates, containers, bootloaders — without changing the client core. Source: [How Mender works](#), [Use an Update Module](#).
- Security posture: device has **no open ports**; all comms are client-initiated HTTPS polling with end-to-end TLS. Source: [How Mender works](#).

§4. Server architecture

- The Mender Server is a **microservices** architecture spread across multiple repositories. Source: [How Mender works](#) and corroborating community/vendor write-ups.
- **API gateway**: Traefik routes device/user/API requests to the appropriate microservice. (Vendor/community-sourced; consistent across multiple writeups — treat the specific gateway = Traefik as **medium confidence**, exact at time of writing.)
- **Message broker**: NATS, used by some microservices for orchestration and remote troubleshooting.
- **Database**: MongoDB for persistent backend state.
- **Artifact storage**: S3 bucket (or S3-API-compatible) or Azure Storage Account.

Note: this stack (microservices + MongoDB + NATS + Traefik) is materially different from Helix's mandated stack (Go + Gin, PostgreSQL relational state, MinIO/S3 blobs, REST-primary). Adopting the Mender server wholesale would conflict with the locked Helix decisions.

§5. A/B updates & delta updates

A/B (dual-rootfs): For OS updates, Mender requires a **dual A/B root filesystem partition layout**. The client runs in the active partition (A), writes the update to the inactive partition (B), and on reboot the bootloader tries B. If boot succeeds, B is committed (made permanent); if it fails, the device automatically falls back to the unchanged A partition — atomic, image-based, power-loss-safe. Reboot downtime is roughly ~60s per Mender's own figure. Source: [How Mender works](#).

This is conceptually the **same robustness model** as Android's native A/B / Virtual A/B, but implemented via Mender's own bootloader integration (U-Boot / GRUB), **not** via Android's `update_engine`. (Mender's open-source licenses list includes `u-boot` under GPLv2 and `grub` under GPLv3, confirming bootloader-side integration — see §9.)

Delta updates: Mender supports binary delta updates, advertised as saving ~70–90% bandwidth (vendor figure — **UNVERIFIED** as an independent benchmark; it is Mender's own marketing claim). Mender 3.6 added **auto-generation of delta updates**, and a later capability adds **server-side generation** so per-device deltas are produced transparently by the server. Source: [Mender 3.6: Auto-generation of delta updates](#), [Server-side generation of delta updates](#). **Important:** delta generation is a **paid feature** (see §9).

§6. Phased deployments & fleet orchestration

Mender supports **phased rollouts** (gradually rolling out an update across a fleet in time-delayed phases), **dynamic grouping** (auto-group devices by inventory attributes), **scheduled deployments**, and **automatic retry** of failed deployments. Source: [Delta updates, phased rollouts, scheduling blog](#), [Pricing/plans](#).

Critically, per the pricing page (consulted 2026-06-07): - **Phased rollouts** — listed as **Enterprise-tier**. - **Dynamic groups & deployments** — **Enterprise-tier**. - **Scheduled deployments** and **automatic retry** — begin at **Professional** tier.

So the orchestration features that most overlap with Helix's intended control plane are **not** in the free open-source offering.

§7. Android support — the reality

This is the decisive finding for Helix.

- **Official device-support docs** list: Debian-family Linux (Debian, Ubuntu, Raspberry Pi OS — officially supported), Yocto Project (via board integration), other Linux via Buildroot/OpenWRT or compile-from-source, and **Zephyr OS** (via the Mender MCU client, Zephyr 4.2). **Android is not mentioned at all.** Source: [Device Support](#) | [Mender docs](#).
- A prior search-surfaced statement of Mender's position is explicit: “**Mender does not yet support Android.**” (Attributed to Mender's FAQ/docs via search; the live device-support page simply omits Android entirely. Treat the exact wording as **medium confidence**, but the substance — no official Android support — is **high confidence** because Android appears nowhere in the supported-platform list.)
- Mender's A/B model is **bootloader-driven (U-Boot/GRUB)**, fundamentally different from Android's `update_engine` + `boot_control` HAL + Virtual A/B. There is **no Mender Android client** and no integration with `update_engine`.

Implication: Mender cannot be “pointed at” an Android 15 device. Using it for Helix would require building an Android client and an Android A/B integration from scratch — i.e., re-implementing the exact thing Helix already plans to do natively. That eliminates Mender's wrap value.

§8. Client language & coupling

- **Client language:** The Mender Client (v3.x line, `mender-client` package) is written in **Go**. Source: search-surfaced Debian/APT package note (“`mender-client` package corresponds to the Mender Client written in Go (version 3.x.y)”). The newer split daemons (`mender-authd`/`mender-updated`) continue the Go client lineage. The client repo is licensed Apache-2.0 (see §9). (Note: a separate C++ rewrite effort has been reported in the Mender ecosystem for newer client versions — **UNVERIFIED** here; not confirmed against a primary source as of 2026-06-07. The 3.x Go client is the verified baseline.)
- **Coupling (client↔server):** Loose at the wire level — pure **HTTPS polling**, client-initiated, no open device ports, REST APIs. The client can run **fully standalone** (no server at all), which demonstrates the update-execution layer is decoupled from the orchestration layer. The **Update Modules** boundary is a clean, executable-based extension point. This means the *client robustness/execution model* is reusable in spirit even if the server is not.
- For Helix, the client being Go aligns with Helix’s Go mandate — but the client is Linux/bootloader-coupled, so the alignment is superficial.

§9. Licensing: open source vs commercial

Client: Released under the **Apache License v2.0** (“All content in this project is licensed under the Apache License v2, unless indicated otherwise”). Source: [mender LICENSE on GitHub](#).

Third-party/bundled components (from the open-source-licenses doc): include Apache-2.0 (e.g. Google Cloud Go libs), **GPLv2** (u-boot), and **GPLv3** (grub) — relevant because the bootloader pieces carry copyleft. Source: [Open source licenses | Mender docs](#).

Server / feature split (open-core model): Mender follows an **open-core** model. There is a free **Open Source** plan, and paid **Basic/Starter**, **Professional**, and **Enterprise** plans (Professional/Enterprise also offered as a hosted SaaS; Enterprise also on-prem). Per the pricing page (consulted 2026-06-07; prices/tiers change frequently — treat exact \$ as **medium confidence**):

Feature	Tier (as listed 2026-06-07)
Core OTA (A/B rootfs, robust update, Update Modules)	Open Source
Scheduled deployments	Professional+
Automatic retry of deployments	Professional+
Delta updates (robust delta)	Professional+
Server-side / automatic delta generation	Enterprise
Phased rollouts	Enterprise
Dynamic groups & deployments	Enterprise
RBAC	Enterprise

Indicative pricing seen: Basic ~\$34/mo (≤ 50 devices), Professional ~\$291/mo (≤ 250 devices), Enterprise custom/sales-quote, no monthly option. Source: [Pricing - Plans | Mender](#). Northern.tech also publishes a separate **Mender Server Enterprise** license doc — i.e., the enterprise server is **not** under the OSS license. Source: [Mender Server Enterprise license | docs](#).

Bottom line on licensing: The piece Helix would most want to study/reuse (the control plane: phased rollouts, grouping, RBAC, delta orchestration) is the **commercially licensed** part. The freely-licensed parts are the client and the basic server.

§10. Fit for Helix

Dimension	Assessment
Android 15 fit	Effectively zero. No Android client, no <code>update_engine</code> integration, Android absent from supported platforms. Would require building Android support from scratch.
Go fit	Client is Go (good in principle); server is Go microservices but with Mongo/ NATS/Traefik, conflicting with Helix's Gin + Postgres + MinIO + REST stack.
Wrapability	Poor. Can't wrap what doesn't run on Android. Server wrap conflicts with locked stack + open-core licensing of the needed features.
Reuse value	Reference-only. Update Modules abstraction, standalone/managed split, dual-A/B commit/rollback semantics, polling-based no-open-ports security model, and REST API shape are worth studying for Helix's own design.
License risk	Apache-2.0 client is fine to learn from; Enterprise server features are proprietary — do not copy. Bootloader bits are GPL.

Recommendation: Do **not** adopt or wrap Mender for Helix. Keep the locked Helix decision (native A/B via AOSP `update_engine` + custom Go control plane). Use Mender as a **design reference** (Update Modules pattern, rollback semantics, control-plane API ergonomics) in `ota_landscape_report.md` / ADR-0001.

§11. Confidence & open questions

Overall confidence: HIGH on the disqualifying facts (no Android support; orchestration features are paid). **MEDIUM** on exact feature-to-tier mapping and pricing numbers (marketing/pricing pages change), and on the server's exact internal component list (Traefik/NATS/Mongo — vendor/community sourced, plausibly current but not pinned to a versioned spec).

Open / UNVERIFIED items needing confirmation: - Exact current pricing and the precise tier boundary for each feature (re-check the live pricing page near decision time). - Whether the newest Mender client is a **C++ rewrite** (reported in the ecosystem) vs the verified Go 3.x baseline — **UNVERIFIED**. - The ~70–90% delta bandwidth savings is a **vendor claim**, not an independent benchmark — **UNVERIFIED**. - The exact wording “Mender does not yet support Android” was surfaced via search, not re-confirmed on a live page; the substantive fact (Android not supported) is confirmed by its absence from the device-support list.

§12. Sources consulted

All consulted 2026-06-07.

- Mender — Device Support: <https://docs.mender.io/overview/device-support>
- Mender — Introduction: <https://docs.mender.io/overview/introduction>
- Mender — How Mender works: <https://mender.io/engineers/how-mender-works>
- Mender — Client installation overview: <https://docs.mender.io/client-installation/overview>
- Mender — Standalone deployment: <https://docs.mender.io/artifact-creation/standalone-deployment>
- Mender — Use an Update Module: <https://docs.mender.io/client-installation/use-an-updatemodule>
- Mender — Pricing / Plans: <https://mender.io/pricing/plans>
- Mender — Open source licenses: <https://docs.mender.io/release-information/open-source-licenses>
- Mender — Mender Server Enterprise license: <https://docs.mender.io/release-information/open-source-licenses/mender-server-enterprise>
- Mender client LICENSE (Apache-2.0): <https://github.com/mendersoftware/mender/blob/master/LICENSE>
- Mender GitHub client repo: <https://github.com/mendersoftware/mender>
- Mender blog — 3.6 auto-generation of delta updates: <https://mender.io/blog/mender-3-6-auto-generation-of-delta-updates>
- Mender blog — Server-side generation of delta updates: <https://mender.io/blog/server-side-generation-of-delta-updates>
- Mender blog — Delta updates, phased rollouts, scheduling: <https://mender.io/blog/manage-and-deploy-over-the-air-ota-software-updates-at-scale>